



MTSU ASQI Lab

Autonomous Systems and Quantum Intelligence Laboratory
Middle Tennessee State University

Unmanned Aerial Operations

Browser-based ground control for the ASQI research drone fleet

Project overview: hardware, software, capability, and where it goes next

22 September 2026

Fleet: Piper (flying) · Omega (built, not yet flown) · Vulkan (in build)

Source: tanzimulalam/ASQI-DRONE (private). This document describes the system as of the date above.

Contents

1. Executive summary	5
Where the project stands	5
What makes it different from a hobby build	5
Where it is going	5
2. What you are doing, in plain words	6
The vocabulary you will hear	7
3. The fleet	7
Why each aircraft must differ in two specific ways	8
Where the fleet is defined	8
4. Hardware	8
Per aircraft	8
Ground station	9
Two limits worth knowing before ordering parts	9
5. The radio link	10
Topology	10
Measured performance	10
The control link failsafe, as it happened	10
6. Software architecture	11
What each program does	11
Ports	12
Three decisions that shape everything else	12
7. How the parts talk to each other	13
Logging in	13
The four message types	13
The fleet API	13
8. The operator interface	14
The fleet screen	14
The cockpit	16
The three banners	16

9. Safety model	17
Layers, from outermost to innermost	17
The failsafes, and what triggers them	17
What has actually been tested	18
10. Flight controller configuration	18
Flight modes on the transmitter switch	19
11. Operating procedures	20
Everyday: fly an aircraft	20
Bench work with the props off	20
Other tools on the aircraft	20
12. What has been achieved	21
Timeline	21
Capabilities today	21
Automated tests	22
13. Lessons that cost real time	22
14. How the code is developed and deployed	23
The repository	23
Getting code onto the machines	23
Working without an aircraft	24
Conventions	24
Where the documentation lives	24
15. Current status and open items	25
Readiness by aircraft	25
Open items, in priority order	25
16. Future direction	26
The goal: three aircraft, one operation	26
The strongest hardware direction: a second, independent radio link	26
Other directions already identified	27
17. Risks	27
18. Quick reference	28
Where things live	28

Commands used most	28
------------------------------	----

1. Executive summary

This project is a **ground control station for research quadcopters, flown from a web browser**. An operator opens a page on a laptop or on the ground station's own screen, sees every aircraft in the fleet with its live health, picks one, types its password, and flies it: on-screen sticks or the keyboard, the aircraft's camera in the middle of the screen, and object detection boxes drawn over the video.

It is not a replacement for Mission Planner or QGroundControl, the standard drone ground stations. Those are general tools for any aircraft. This is a purpose-built cockpit for these specific airframes, built so that the lab can put its own software in the control path and extend it: computer vision today, multi-aircraft coordination next.

Where the project stands

	State	What that means
Piper	Flying	First flight 2026-08-06. Flies from the browser, holds position in Loiter under GUI control, with live video and detection.
Omega	Built and configured	Second airframe, brought up 2026-09-19/20. Reachable from the ground station, answers health checks, has never flown.
Vulkan	In build	Third airframe. Listed on the fleet screen so the fleet is complete, with no computer or address yet.

The ground station itself is finished enough to use daily: it boots into the fleet screen with no manual steps, shows which aircraft are online, checks their health without disturbing them, and hands the operator a cockpit for whichever one they pick.

What makes it different from a hobby build

- **The safety logic lives on the aircraft, not in the browser.** A program on the drone's own computer re-checks every command, refuses control input in the wrong flight mode, confirms the aircraft is armed by asking the flight controller rather than by trusting a button press, and lands the aircraft by itself if the radio link to the ground goes quiet.
- **Video and vision cannot interfere with flying.** They travel on separate channels. If the camera or the detector dies, the aircraft still flies normally.
- **Object detection runs on the ground,** not on the aircraft, so the drone carries no computing load for it.
- **The browser never stores a password.** The operator types it once per session; the ground station holds it and the aircraft decides whether it is right.

Where it is going

The stated goal is a **coordinated swarm of three aircraft**. Everything built so far is a step toward that: one aircraft flying reliably, a second one identical to it, a fleet screen that treats aircraft as a list rather than a single hard-coded address, and a login that can only ever connect to the aircraft the operator chose. The remaining work is covered in section 16.

READ THIS FIRST

Three things in this document are known to be unverified, and are marked where they appear: the battery cell count has never been physically confirmed, the control direction check has only ever been completed for the pitch axis, and no range test has been run. Each one is a safety-relevant gap, not a paperwork gap.

2. What you are doing, in plain words

If someone asks what this project is, this section is the answer.

The problem

A drone normally flies from a hand-held transmitter, and is monitored from a laptop running a general-purpose ground station program. That arrangement is fine for flying, but it is a closed loop: there is nowhere natural to put the lab's own software, and nothing in it understands more than one aircraft at a time as a single operation.

The approach

Put a small computer on the aircraft next to the flight controller. Let that computer own the link to the flight controller and enforce the safety rules. On the ground, run a server that serves a web page, holds the session, and relays the operator's input to the aircraft fifty times a second. The operator's interface is then just a web page, which means it can be improved quickly, can show anything the lab wants it to show, and can be opened from any machine on the network.

Your role in it

You inherited this project with no handover from the previous developer, and you are not the original author of the code. Since taking it on you have: found and fixed the reason stick input had never reached the aircraft, flown it, built a second airframe from a bare flight controller, consolidated two repositories into one, replaced the single-aircraft login with a fleet screen, and fixed a class of bug that silently stopped the aircraft being flown when the browser tab lost focus. The recurring discipline in this project, and the reason those were found, is **reading the vehicle instead of trusting the documentation**.

What it demonstrates

- Real-time control over a wireless link, with a measured failure mode and a failsafe that has fired in flight and behaved correctly.
- A safety architecture with the trusted component placed on the aircraft, where it cannot be bypassed from the ground.
- Embedded GPU inference (NVIDIA Jetson, TensorRT) on a live video stream.

- A fleet model that scales past one aircraft, which is the precondition for swarm work.

The vocabulary you will hear

Term	What it means here
Flight controller	The Pixhawk 6C board that actually stabilises and flies the aircraft. It runs ArduCopter, an open-source autopilot.
Companion computer	The NVIDIA Jetson on the aircraft. It does not fly the aircraft; it talks to the flight controller and to the ground.
MAVLink	The standard message language between a ground station and a flight controller.
Arming	Enabling the motors. An aircraft that is disarmed cannot spin its props.
Pre-arm check	A refusal by the flight controller to arm until conditions are right (GPS, battery, radio, and so on).
Failsafe	Automatic protective action when something is lost: the radio link, the battery, or the operator.
Loiter / Alt Hold / Stabilize	Flight modes. Loiter holds position using GPS, Alt Hold holds height only, Stabilize holds attitude only.
Telemetry	The stream of live readings from the aircraft: battery, GPS, mode, attitude, link age.

3. The fleet

Three airframes of the same design, at three different stages. The ground station treats them as a list, so a fourth would be a configuration change rather than a code change.

	Piper	Omega	Vulkan
Status	Flown since 2026-08-06	Built, configured, never flown	Airframe in build
Airframe	Holybro X500 class	Holybro X500 class	Holybro X500 class
Flight controller	Pixhawk 6C	Pixhawk 6C, ArduCopter 4.6.3	Not yet fitted
MAVLink system id	1	2	Not assigned
Companion computer	Jetson Orin Nano Super, JetPack 6.2	Jetson, JetPack 5.1.3, Python 3.10 built from source	Not yet fitted
Camera	Logitech Brio 101, streaming	Not yet running	Not yet fitted
Network	Hosts its own Wi-Fi hotspot (10.42.0.1)	Campus ethernet only (10.130.143.5)	None
Transmitter	Its own, bound	Its own, separately bound	None

Why each aircraft must differ in two specific ways

- **A unique MAVLink system id.** Two aircraft reporting the same id cannot be told apart by a ground station. Piper is 1, Omega is 2.
- **Its own bound transmitter.** One transmitter bound to two receivers commands **both** aircraft at once. Model memory on the transmitter does not prevent this, because the radio identity belongs to the transmitter, not to the model slot.

Where the fleet is defined

On the ground station, in a single file, `/etc/drone/fleet.json`. It lists each aircraft's name, network address, and the token used for read-only health checks. An aircraft still being built is listed with no address at all:

```
{
  "drones": [
    {
      "name": "Piper",
      "ip": "10.42.0.1",
      "token": "...",
    },
    {
      "name": "Omega",
      "ip": "10.130.143.5",
      "token": "...",
    },
    {
      "name": "Vulkan",
      "ready": false
    }
  ]
}
```

The file is readable only by the ground station's own user and root. The tokens in it are used only to ask an aircraft how it is; they are never sent to a browser, and an automated test enforces that.

WHY THE AIRCRAFT BEING FLOWN IS NEVER HEALTH-CHECKED

The program on each aircraft sends its telemetry to whoever last contacted it with a valid packet. If the ground station health-checked an aircraft that someone was flying, the telemetry stream would follow the health check and the cockpit would go quiet in mid-air. The ground station therefore refuses to probe an aircraft that has an open session, and shows that session's live readings on its card instead.

4. Hardware

Per aircraft

Component	Part	Notes
Airframe	Holybro X500 class quadcopter	Roughly 500 mm between diagonal motors
Flight controller	Holybro Pixhawk 6C	Runs ArduCopter. Connects to the Jetson over USB as <code>/dev/ttyACM0</code>
Companion computer	NVIDIA Jetson Orin Nano Super	Runs the control daemon and the camera daemon
Camera	Logitech Brio 101 (USB)	Streamed as MJPEG over HTTP

Component	Part	Notes
Radio	TP-Link Archer T2U PLUS (USB, 2.4 GHz)	Hosts the aircraft's own Wi-Fi network at 20 dBm
Battery	3300 mAh pack, thresholds set for 4S	Cell count never physically confirmed. Verify before trusting any battery failsafe value
Transmitter	FlySky FS-i6X with FS-IA6B receiver	Manual backup control. Receiver feeds the flight controller over PPM

Ground station

Component	Part	Notes
Computer	NVIDIA Jetson Orin Nano Super, JetPack 6.2	Runs the web bridge and the detector. Has a full desktop, which matters (see section 13)
Radio	USB Wi-Fi adapter with an external antenna	Replaced the Jetson's built-in radio. The drone hears it about 11 dB better
Operator laptop	Windows, any machine on the network	Opens the same web page. A desktop shortcut finds the right address

THE RADIO UPGRADE WAS THE HIGHEST-VALUE CHANGE AVAILABLE

The link was measured as lopsided: the ground heard the aircraft at -74 dBm while the aircraft heard the ground at -86 dBm. The weak direction is the one carrying stick input, and that asymmetry caused a real in-flight failsafe on 2026-08-06. The Jetson's built-in radio is capped in hardware at 15 dBm; it accepts a request for 20 dBm, reports success and ignores it. The USB adapter with a real antenna closed most of that gap, measured at 11 dB better as heard by the aircraft. The gain is the antenna, not transmit power.

Two limits worth knowing before ordering parts

- **Transmit power is capped at 20 dBm** to stay within FCC limits. The Realtek driver will accept and report 30 dBm without regard to the regulatory database. This is university equipment; do not raise it.
- **The receiver is wired for PPM, which carries only 8 channels.** The FS-i6X is a 10-channel transmitter, so channels 9 and 10 never reach the flight controller. A planned feature (detecting that a safety pilot has grabbed the sticks, per axis) needs those channels and therefore cannot work on this wiring. Switching the receiver to i-BUS would recover them.

5. The radio link

Everything the operator does crosses one 2.4 GHz Wi-Fi link. Understanding its shape explains most of the project's operational rules.

Topology

The aircraft hosts the network and the ground station joins it. This is deliberate. Campus Wi-Fi isolates clients from each other, which silently blocks the traffic between ground and aircraft, so a shared network cannot work. On the bench, wired ethernet is used instead; in the field, the aircraft's own hotspot.

Context	Addressing
Bench, wired	Ground station 10.131.237.193, Omega 10.130.143.5
Flight	Piper hosts SSID drone-tx on 2.4 GHz channel 11 and is 10.42.0.1

Measured performance

Measurement	Value	Comment
Aircraft to ground	39.8 Mbit/s	Carries video and telemetry
Ground to aircraft	28.8 Mbit/s	Carries stick input
Video consumes	10.1 Mbit/s	Roughly a quarter of the strong direction
Signal, bench	about -40 dBm	Close range, line of sight
Signal, outdoors	-56 dBm	Both machines outside
Signal, through a wall	-71 dBm	A 15 dB penalty, larger than any hardware upgrade would buy
Detector frame rate	about 30 fps wired, 20 fps over Wi-Fi	Frame starvation, not GPU load; inference stays at 18 to 22 ms

FLY WITH THE GROUND STATION OUTSIDE AND UNTETHERED

On the day of the first flight the ground station was carried indoors to reach a wired ethernet port so a remote observer could watch. That single wall cost 15 dB, more than any radio change available. Read the logs afterwards instead.

The control link failsafe, as it happened

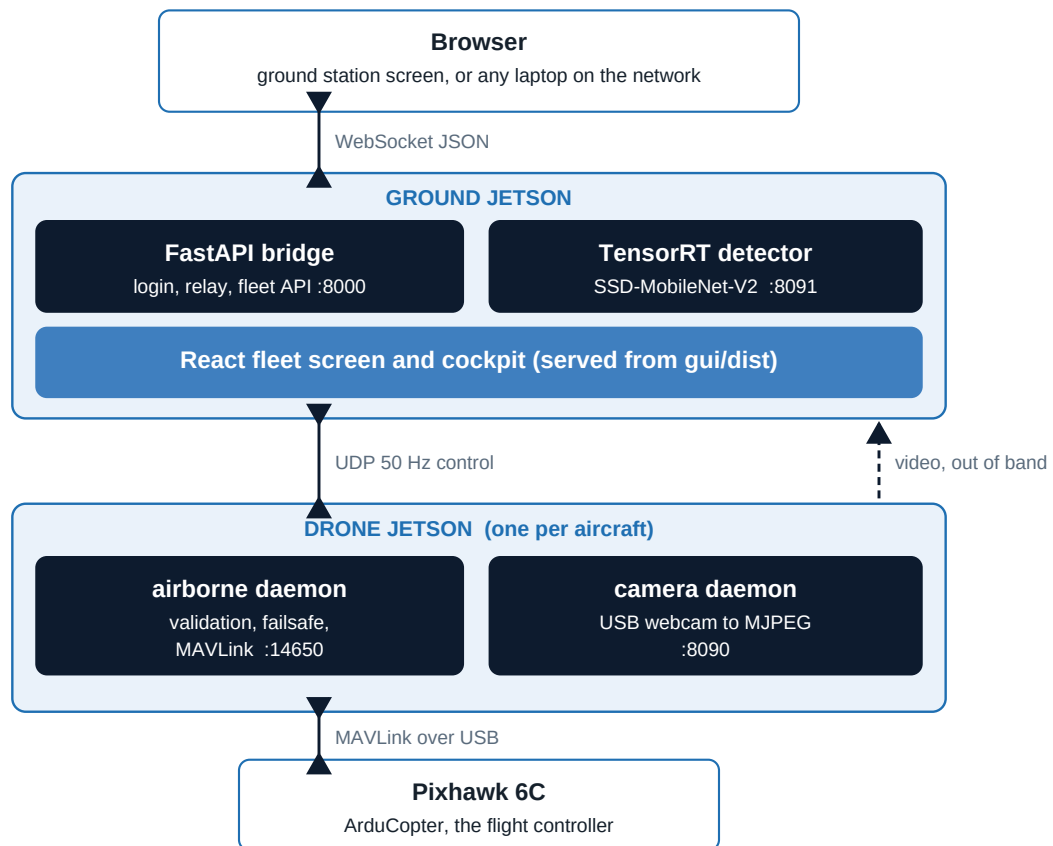
```
14:45:58 vehicle armed
14:46:45 FAILSAFE: control link lost (616 ms) -> commanding LAND
14:46:46 failsafe latch cleared: vehicle disarmed
```

Forty-seven seconds into the first flight, at altitude, the path from ground to aircraft went silent for longer than the 600 ms threshold. The aircraft neutralised the stick input, released its hold on the controls, and commanded a landing. **Nothing malfunctioned.** This is the failsafe doing its job, and it remains the only time it has fired outside a test. The most likely cause has since been identified and fixed: a browser tab that is not in front stops sending, which is covered in section 13.

A 50 m range test has still never been run. It is the one original goal of the project that remains outstanding, and the tool for it (ground/tools/linkcheck.sh) is written and waiting. Watch the negotiated transmit rate rather than the signal strength: the rate degrades first.

6. Software architecture

Five programs, on three machines. Each one does a single job, and the boundaries between them are where the safety guarantees live.



The ground station never talks to the flight controller directly. Every command passes through the aircraft's own daemon, which can refuse it.

What each program does

Program	Runs on	Job
Bridge (ground/app)	Ground Jetson	Serves the web page, holds the login session, relays operator input to the aircraft 50 times a second, and answers the fleet API
Cockpit (ground/gui)	Browser	The fleet screen and the cockpit. React, built with Vite, served as static files
Detector (ground/detector)	Ground Jetson	Pulls the aircraft's video, runs SSD-MobileNet-V2 on TensorRT, and publishes boxes
Airborne daemon	Each drone Jetson	The safety boundary. Validates every packet, runs the failsafe state machine, speaks MAVLink to the flight controller
Camera daemon	Each drone Jetson	Finds the USB webcam and serves it as MJPEG. Not flight critical

Ports

Port	Machine	Purpose
8000	Ground	Web page, WebSocket, fleet API, camera proxy
8091	Ground	Detector results
8090	Each aircraft	Camera MJPEG stream
14650	Each aircraft	Control, commands and heartbeats (UDP)

Three decisions that shape everything else

- **The aircraft is the safety boundary.** The ground station is a convenience; the daemon on the aircraft is the authority. It re-validates every packet, refuses control input outside approved flight modes, confirms arm state from the vehicle rather than from a button press, and runs its own link-loss failsafe.
- **Video and detection are out of band.** They use separate connections, so neither can stall the 50 Hz control path, and both can be dead while the aircraft flies normally.
- **Detection runs on the ground.** The ground Jetson pulls the video and does the inference, so the aircraft carries no computing load for vision at all.

7. How the parts talk to each other

The full contract is in the repository as `airborne/PROTOCOL.md`. This is the shape of it.

Logging in

The operator presses Fly on an aircraft and types its password. The ground station sends that password to that one aircraft and waits to see whether it answers. If it does, the password was right, and the cockpit opens. The password never reaches the browser's storage, and the ground station keeps no list of valid passwords: the aircraft itself is the authority.

A REAL BUG WORTH UNDERSTANDING

Originally the ground station judged a password correct by probing and seeing whether telemetry arrived. But the aircraft streams telemetry to whoever last contacted it, so after any one correct login, later probes on the same connection got an answer no matter what password they carried. **Every password was accepted**, giving an operator a cockpit with live video and telemetry and no control authority at all. The fix is to probe from a brand new connection each time, and a test now fails if anyone undoes it.

The four message types

Message	Direction	Rate	Contents
Control	Ground to aircraft	50 Hz	Four stick axes, each between -1 and +1, a sequence number, and the token. Out-of-range or out-of-order packets are rejected whole
Command	Ground to aircraft	On action	Arm, disarm, set flight mode, take off, manual failsafe, resume
Heartbeat	Ground to aircraft	1 Hz	Keeps telemetry flowing when nobody is flying. Does not count as control input for the failsafe
Telemetry	Aircraft to ground	10 Hz	Battery, GPS, mode, arm state, altitude, speed, EKF health, link ages, failsafe state, pre-arm messages

Video is deliberately not part of this. It is an ordinary HTTP stream on its own port, reverse-proxied by the ground station so the browser never needs the aircraft's address.

The fleet API

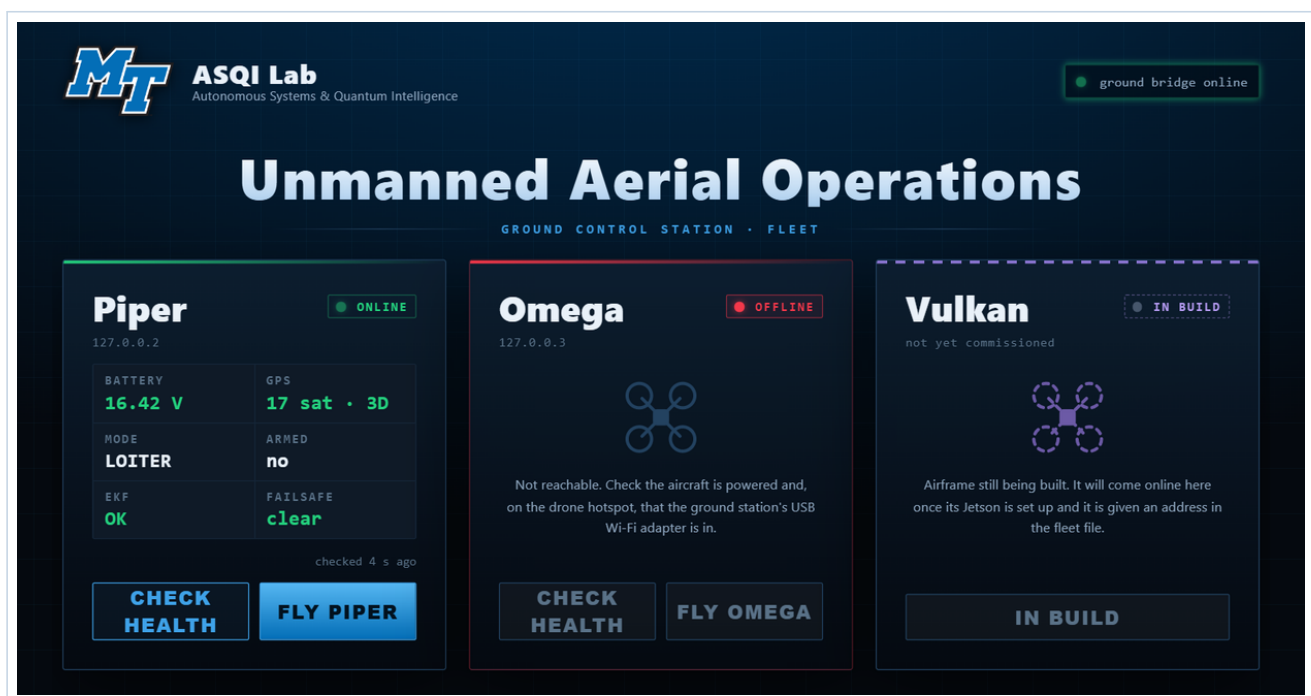
Call	What it does
GET <code>/api/fleet</code>	Every aircraft with its reachability, whether it is in a session, and its last known health. Reachability is a plain network connection test to the drone's computer, so polling it never touches the control port

Call	What it does
POST /api/fleet/<name>/health	Asks one aircraft for a fresh reading. Refused for an aircraft still in build, and never sent to an aircraft that is being flown

8. The operator interface

The fleet screen

This is what opens when the page loads. One card per aircraft, refreshed continuously.

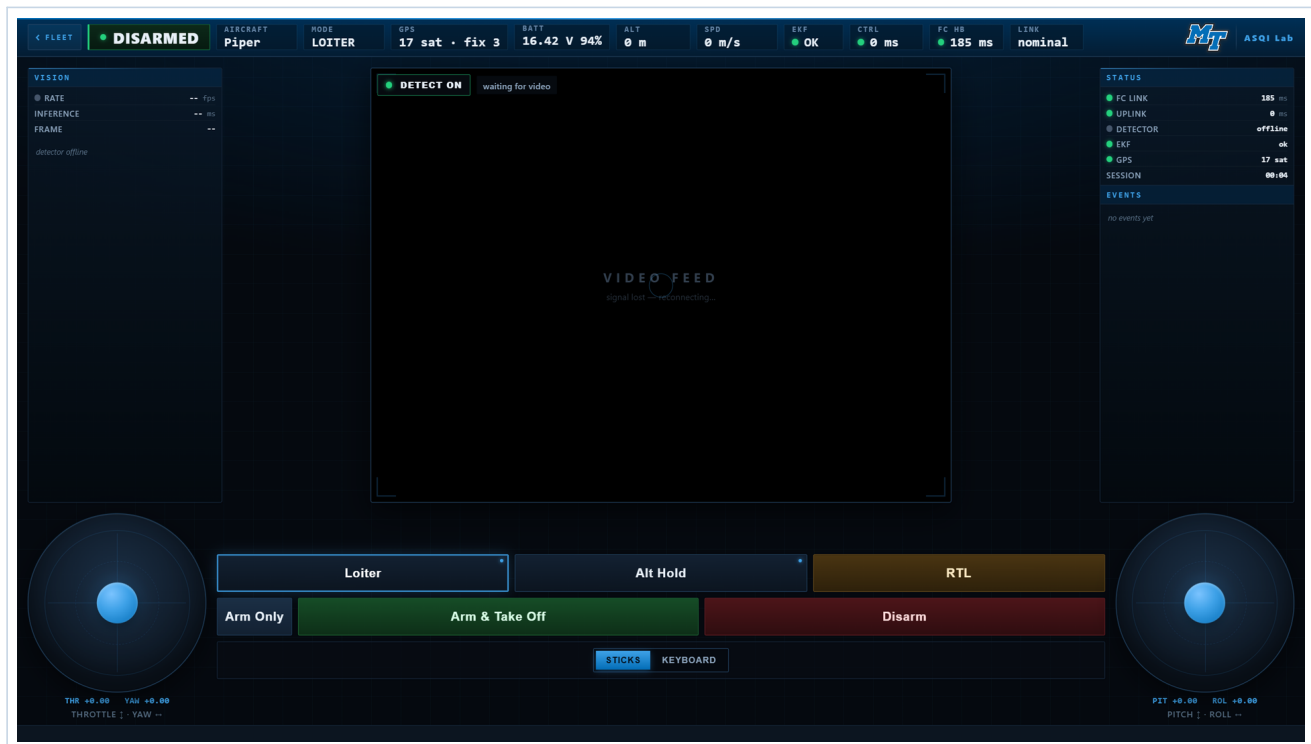


The fleet screen. Piper online with live health, Omega offline in red, Vulkan shown as in build.

Badge	Meaning
ONLINE (green)	The aircraft's computer answers. Health is read when it comes online and whenever Check health is pressed
OFFLINE (red)	No answer. Powered down, or not on a network the ground station can reach
IN SESSION (blue)	Being flown right now. The card shows the live session readings rather than probing the aircraft
DAEMON SILENT (amber)	The computer is up but its flight program did not answer
IN BUILD (violet)	Listed for completeness. No address, cannot be flown

Each card shows battery voltage, GPS satellites and fix quality, flight mode, arm state, EKF (position estimate) health, failsafe state, and the most recent pre-arm message. A battery reading below 1 V is shown as *not connected*, which means the flight controller is running on USB power with no pack plugged in.

The cockpit



The cockpit. Telemetry across the top, video with vision and status panels either side, and the flight controls below. Shown against a stand-in aircraft, so the video panel is empty.

- **Telemetry band.** Arm state, which aircraft, flight mode, GPS, battery, altitude, speed, EKF, and three separate link ages. Each reading reserves its worst-case width so the row never reflows and never resizes the video beneath it.
- **Fleet button, top left.** Ends the session and returns to the fleet screen. Disabled while the aircraft is armed, so it cannot be hit by accident in flight.
- **Sticks.** Self-centering on all four axes. Releasing the left stick returns throttle to centre, which in Alt Hold, Loiter and PosHold means hold altitude.
- **Keyboard flying.** W and S for throttle, A and D for yaw, numpad 8/2 for pitch and 4/6 for roll. Input eases in over about 300 ms and is capped at 60% of full deflection. Losing window focus with a key held zeroes every axis immediately.
- **A blue dot on a mode button** means the aircraft accepts stick input in that mode. Outside those modes it refuses the input outright, which would otherwise be invisible until a stick did nothing.
- **Arm Only versus Arm and Take Off.** Arm Only arms and stops there. Arm and Take Off arms and commands a climb, which must never be used with the props off.

The three banners

Banner	Meaning
NO CONTROL AUTHORITY	The aircraft is receiving packets and discarding them, usually a password mismatch. Everything else still looks healthy, which is exactly why this needs a banner
FAILSAFE ACTIVE	The aircraft has latched a failsafe and is landing. Clears only on Resume or disarm
PILOT TAKEOVER	A safety pilot moved the transmitter sticks. The on-screen controls stay inactive until Resume

9. Safety model

The core principle: **the ground station is a convenience, and the aircraft is the authority.** Everything else follows from that.

Layers, from outermost to innermost

Layer	What it enforces
Browser	Nothing. It is untrusted by design and holds no password
Ground bridge	Login, message shape, and injecting the token. It cannot force the aircraft to do anything
Airborne daemon	The boundary. Token on every packet, axis ranges, packet ordering, allowed flight modes, arm state confirmed from the vehicle, and the link-loss failsafe
ArduCopter	Its own independent pre-arm checks, geofence, battery failsafe, radio failsafe and ground-station failsafe
Transmitter	A human with a physical radio who can take the aircraft back through the mode switch

The failsafes, and what triggers them

Failsafe	Trigger	Action
Control link	No valid control packet for 600 ms	Neutralise sticks, release control, command LAND. Latches until disarm or Resume
Ground station (ArduCopter)	Its own view of the link, independent of our software	Configured on the vehicle
Battery low	Pack at or below 14.0 V	Return to launch
Battery critical	Pack at or below 13.6 V	Land
Radio	Transmitter signal lost	ArduCopter's own radio failsafe
Geofence	300 m radius or 100 m ceiling breached	Return to launch

Failsafe	Trigger	Action
Pilot takeover	Safety pilot moves the sticks	On-screen control is suspended until Resume

THE ONE RULE THAT IS NEVER RELAXED

Arming indoors requires turning off the pre-arm checks and the geofence (ARMING_CHECK=0 and FENCE_ENABLE=0). **Both must be set back to 1 before the aircraft goes outside, and the values must be read back from the vehicle to confirm it.** Flight controller settings survive power cycles: turning the aircraft off does not restore them. On 2026-08-06 a restore silently failed on a library crash and the aircraft was minutes from flying with every pre-arm check disabled. This is why the tool that does it reads the values back and why props stay off the whole time the relaxations are in place.

What has actually been tested

Test	Status
Control-link failsafe in the air	Happened involuntarily on 2026-08-06 and behaved correctly. Never deliberately tested on the bench
Control direction check, all four axes	Only pitch. It was found reversed in the air, not on the bench. Roll, yaw and throttle have never been verified props-off
Motors respond individually, props off	Done on both aircraft
Cold boot with nothing started by hand	Verified on both ground and drone computers
50 m range test	Never run

10. Flight controller configuration

Read off Piper on 2026-08-05. **Re-read before trusting any of it**, using `airborne/tools/params.py`, which confirms every write by reading it back from the vehicle. Three of six safety settings once contradicted the documentation, which is why this warning exists.

Setting	Value	Meaning
SYSID_MYGCS	250	Must match the daemon. If it does not, stick input is silently discarded
ARMING_CHECK	1	All pre-arm checks on
FS_GCS_ENABLE	1	Flight controller runs its own ground-station failsafe
FENCE_ENABLE	1	Geofence on: 300 m radius, 100 m ceiling, action return to launch

Setting	Value	Meaning
FS_THR_ENABLE	1	Throttle failsafe on
RC_OVERRIDE_TIME	3	Transmitter regains authority 3 s after the daemon stops
BATT_MONITOR	4	Voltage and current monitoring
BATT_CAPACITY	3300	mAh
BATT_ARM_VOLT	14.0	Refuses to arm below this
BATT_LOW_VOLT / ACT	14.0 / RTL	Low battery returns to launch
BATT_CRT_VOLT / ACT	13.6 / Land	Critical battery lands

Flight modes on the transmitter switch

The mode switch is channel 5, which is switch SwC on the FS-i6X. A three-position switch reaches only slots 1, 4 and 6.

Switch position	Mode	What it does
Up	STABILIZE	Attitude only. Throttle maps straight to motor output
Middle	ALT_HOLD	Holds height using the barometer. Needs no GPS
Down	LOITER	Holds position using GPS

TWO ARMING REFUSALS THAT LOOK LIKE FAULTS AND ARE NOT

“Need Position Estimate” is a property of the flight mode, not a pre-arm check. Loiter, PosHold, Guided and RTL each require a position estimate by virtue of being that mode, so turning the checks off does not help. Alt Hold needs only the barometer, which is why it is the mode for bench work.

“High GPS HDOP” blocked arming once with 16 satellites and a healthy 3D fix, because HDOP measures satellite *geometry*, not count. Satellites bunched in one part of the sky give a poorly conditioned solution. Wait, or move away from whatever is blocking the low horizon. Do not lower the threshold: that arms on a position the flight controller has said it does not trust.

11. Operating procedures

Everyday: fly an aircraft

Nothing is started by hand. All services come up on boot on all machines.

- Power the ground station and the aircraft.
- Open the ground station page, or use the **ASQI Ground Station** shortcut on the operator laptop, which tries the wired address then the hotspot address.
- Check the aircraft's card: online, battery sensible, GPS satellites and fix, no pre-arm message that matters.
- Press Fly, enter that aircraft's password.
- Turn on that aircraft's transmitter, throttle fully down, and press the hardware safety switch until its light is solid.
- Select a flight mode, arm, fly. Press Fleet to end the session when finished.

Bench work with the props off

PROPS OFF, AIRCRAFT SECURED

An unloaded quadcopter vibrates and will walk off a bench. Use **Arm Only**, never Arm and Take Off. Use **Stabilize** for direction checks, driven from the transmitter: in Alt Hold the height controller keeps raising thrust looking for an altitude response it can never get, ramps the motors toward saturation, reports *Potential Thrust Loss*, and refuses to disarm because a centred throttle means hold altitude. Pull throttle fully down to disarm.

To arm indoors, the pre-arm checks and geofence have to come off. One command does it and reads the values back:

```
airborne/tools/benchmode.sh on      # relax, props off only
airborne/tools/benchmode.sh off    # restore, required before flight
airborne/tools/benchmode.sh status # read both, change nothing
```

The script stops the daemon (which holds the serial port), writes the values, reads them back from the vehicle, and restarts the daemon on every exit path including failure. **Do not fly until the read-back shows 1 for both.**

Other tools on the aircraft

Tool	Purpose
params.py	Read and write flight controller settings, with read-back confirmation
motortest.py	Spin each motor individually, props off, to confirm wiring and order

Tool	Purpose
<code>rcwatch.py</code>	Watch live transmitter channel values, for checking calibration and switches
<code>fcreboot.py</code>	Reboot the flight controller without unplugging it
<code>rangeLog.sh</code>	Log link quality from the aircraft side
<code>linkcheck.sh</code> (ground)	One line per second of signal, rate, ping and video throughput, for the range test
<code>sim_drone.py</code> (ground)	A stand-in aircraft for working on the interface with no drone present

12. What has been achieved

Timeline

When	What
2026-08-04/05	The whole system read off the hardware for the first time. Found that stick input had never reached the aircraft, that every pre-arm check was disabled, and that both battery failsafes did nothing. All fixed
2026-08-05	Cold-boot verification: every service starts on its own on both machines
2026-08-06	First flight. GPS locked with 16 satellites, held position in Loiter under browser control. Pitch found reversed in the air and corrected. The control-link failsafe fired for real and behaved correctly
2026-09-11	Transmitter, receiver and mode switch fully mapped. Found the receiver was not in PPM mode and that a committed fix had never been deployed to the aircraft
2026-09-17	Flown again. Motor and direction checks with props off
2026-09-19/20	Omega built from a bare flight controller: frame, calibration, safety settings, compass priority, its own token and services
2026-09-20	Two repositories consolidated into one private repository with both histories preserved
2026-09-21	Fleet screen shipped: all three aircraft, live health, per-aircraft login, Fleet button, and a fix for sessions that never ended

Capabilities today

Capability	State
Fly one aircraft from a browser, sticks or keyboard	Working, flown
Live video with object detection boxes	Working: SSD-MobileNet-V2, TensorRT, 18 to 22 ms per frame, about 30 fps wired
Fleet view with live health of every aircraft	Working

Capability	State
Per-aircraft login that cannot connect to the wrong aircraft	Working, with tests
Link-loss failsafe that lands the aircraft	Working, proven in flight
50 Hz control that survives a background browser tab	Fixed: the sender was moved off browser timers into a worker
Everything starts on boot, on every machine	Verified
Second aircraft ready to fly	Built and configured, first flight pending
Two aircraft flown at once	Not built yet

Automated tests

Suite	Result
Airborne daemon	81 pass: stick scaling, packet validation, failsafe state machine, camera, controller
Ground bridge	All pass except 3 known stale tests that predate the login gate and are tracked separately
Fleet and login security	18 pass, including: tokens never reach a browser, the aircraft being flown is never probed, and a session always ends when the last browser leaves

13. Lessons that cost real time

Twenty of these are recorded in the project's handoff document. These are the ones that changed how the system is built or operated.

What happened	What it means now
Stick input had never reached the aircraft. ArduPilot discards control from any system id other than the one it expects. The vehicle expected 255; the daemon sent 250. Arm and mode commands are accepted from anyone, so those worked, which made it look like an interface bug	Any second ground station or radio added later must have its system id decided deliberately, or this returns
Any password was accepted. The login check was satisfied by telemetry that was already flowing for unrelated reasons	Login probes run on a brand new connection each time, enforced by a test

What happened	What it means now
A background browser tab stopped flying the aircraft. Hidden tabs have their timers throttled: 0 to 3 control packets in 2 seconds against the 50 needed. The aircraft failsafes within a second	The sender was moved off browser timers into a worker, and the interface now says when it has stopped transmitting
Pitch was reversed, and it was found in the air. Forward on screen commanded nose up	Direction checks belong on the bench with props off, on all four axes. This is still outstanding for roll, yaw and throttle
A safety parameter restore failed silently when a library crashed mid-write, leaving the aircraft with all checks disabled minutes before flight	Never report a setting as written without reading it back. The bench tool now does this automatically
A committed fix was never deployed. The aircraft was five weeks and two commits behind, and failed in exactly the documented way	Documentation describes the repository, not the machines. Check what a machine is running before trusting anything written about it
The battery percentage is a coulomb counter, not a charge estimate. It resets to about 100% on every power-up and once read 98% on a pack at roughly a third	Trust voltage, not percentage
Compass slots reshuffled on reboot, making an uncalibrated internal compass primary and producing a calibration error on a compass that was calibrated	Set compass priority by device id, not by disabling slots
The detector lost a memory race at boot and left its port open with nothing behind it, while the service looked healthy	Close the browser and the updater on the ground station before flying. The launcher now waits for free memory

14. How the code is developed and deployed

The repository

One private repository, tanzimulalam/ASQI-DRONE, with two halves: ground/ for the ground station and airborne/ for the aircraft. They run on different machines and deploy independently, but they change together, so they live together. It consolidates two earlier repositories with both histories preserved.

Getting code onto the machines

Neither Jetson can reach GitHub: the aircraft have no internet in flight configuration, and the repository is private. Changes travel as a **git bundle** over SSH, which is a single file containing the commits the machine does not yet have. The ground station also cannot build the web interface, because its Node.js is too old, so the build is done on a laptop and copied across.

THE RULE THAT CAME OUT OF THIS

Before updating or diagnosing a machine, ask it what it is running: which directory its service uses, and which commit is checked out there. Twice in this project a problem was diagnosed against the repository rather than against the machine, and both times the machine was running something older.

Working without an aircraft

A stand-in aircraft (`ground/tools/sim_drone.py`) answers the login and streams steady telemetry on a local address, so the fleet screen and cockpit can be developed on any laptop. It has no flight logic; it is for interface work. The screenshots in this document were taken against it, which is why the video panel is empty.

Conventions

- A behaviour change comes with tests; a bug fix comes with a test that fails without it.
- Anything that crosses the link between ground and aircraft updates the protocol document in the same commit.
- Flight controller settings are written with the tool that reads them back, never by hand.
- Work on a branch and merge when it has been checked on hardware, or say in the commit message that it has not been.

Where the documentation lives

Document	Read it when
<code>ground/HANDOFF.md</code>	You are picking the project up cold. Architecture, settings read off the vehicles, twenty documented traps, flight logs, and the fleet. Its first section is written for an AI assistant joining the project
<code>README.md (root)</code>	You want the overview, the layout, and how to run or deploy anything
<code>ground/README.md</code>	You are going to fly, bench test or range test
<code>airborne/README.md</code>	You are changing the daemon or bringing up a new aircraft
<code>airborne/PROTOCOL.md</code>	You are changing anything that crosses the link

15. Current status and open items

Readiness by aircraft

	Piper	Omega	Vulkan
Flies	Yes	Not yet	No
Ground station sees it	Yes	Yes	Listed as in build
Camera and detection	Working	Not set up	No
Own Wi-Fi hotspot	Yes	No, campus ethernet only	No
Blocking items	Direction check on roll, yaw and throttle; battery cell count unconfirmed	First flight; camera; a network it can fly on	Build; computer; flight controller; calibration

Open items, in priority order

#	Item	Why it matters
1	Confirm the battery cell count physically	Every battery failsafe threshold depends on it, and it has never been checked
2	Direction check on roll, yaw and throttle, props off	Pitch was found reversed in the air. The other three axes have never been verified
3	Deliberate link-loss test on the bench	It has happened by accident in flight, which is not the same as testing it
4	50 m range test	The one original goal never met. The tool exists
5	Omega's first flight	The second aircraft is otherwise ready
6	A network layout that reaches every aircraft at once	Today each aircraft hosts its own hotspot, so the ground station can only be on one at a time. This is the blocking problem for the swarm
7	Omega's camera daemon	Needs the vision libraries in its Python environment
8	Stronger passwords before public demonstrations	Anyone on the aircraft's Wi-Fi who guesses the password gets control
9	Read Piper's firmware version, and confirm where its code runs from	Never checked since the repositories were merged
10	Bring Omega to the same JetPack version as Piper	Needs an x86 Ubuntu machine to reflash. Until then the fleet is not uniform

16. Future direction

The goal: three aircraft, one operation

The end state is a coordinated swarm of Piper, Omega and Vulkan. That is a series of steps rather than a single project, and each one is useful on its own.

Phase	What it means	What it needs
A. Two aircraft, one at a time	The ground station can fly either aircraft, though only one per session	Done. This is what the fleet screen delivered
B. Two aircraft reachable at once	Both aircraft on one network with the ground station, health of both live while one is flown	A network decision: a ground-hosted access point, or a small field router, instead of each aircraft hosting its own
C. Two aircraft flown at once	One operator, two cockpits, or one screen with both aircraft	Bridge sessions keyed per aircraft, and an interface that makes it obvious which aircraft input is going to
D. Coordinated flight	Aircraft holding formation or dividing an area, with the operator commanding the group rather than each vehicle	Position sharing between aircraft, a group command layer, and a safety model for what happens when one aircraft drops out
E. Vision in the loop	Detection results influencing flight: following, searching, or covering an area	The detector output feeding a decision layer rather than only the screen

The strongest hardware direction: a second, independent radio link

Today one Wi-Fi link carries everything: control, telemetry and video. A long-range radio link (mLRS, an open-source system supported by ArduPilot) would carry control and telemetry over a separate band at far greater range, leaving Wi-Fi to carry only video.

Option	Description	Verdict
1	Radio receiver into the flight controller, telemetry only	What the system is designed for. Gives an independent path that survives total Wi-Fi loss. Start here
2	Radio receiver into the aircraft's Jetson	Architecturally better: keeps the daemon as the only commander. The catch is bandwidth, since 50 Hz control is roughly 44 kbit/s, which is a lot for a long-range radio. Needs a compact encoding first

Option	Description	Verdict
3	Move all control onto the radio link	A genuine redesign, with the failsafe logic re-sited. Scope it separately

Buy the **900 MHz** variant, not 2.4 GHz: the 2.4 GHz version would share a band with the video hotspot, which is the problem rather than the fix. Both a transmitter and a receiver module are required; some listings sell only one. And whichever option is chosen, decide its MAVLink system id deliberately, or the silent-sticks failure returns.

Other directions already identified

- **Cellular link.** A modem on the aircraft has been discussed. Note that the operating system's modem manager grabs the flight controller's serial port; the current workaround disables it entirely, which stops working the moment a real modem is fitted. The proper fix is a rule telling the modem manager to ignore the flight controller.
- **Receiver on i-BUS instead of PPM**, recovering two transmitter channels and making per-axis pilot takeover possible.
- **A lighter ground station session**, so a student opening a browser cannot starve the detector of memory.
- **Mission-level flying:** waypoints and automatic patterns, rather than only direct stick control.

17. Risks

Risk	Exposure	What reduces it
Unverified battery cell count	Every battery failsafe threshold could be wrong for the actual pack	Physically confirm the pack, then re-check the thresholds
Three axes never direction-checked	A reversed axis discovered in flight, as already happened once with pitch	A props-off check on all four axes before the next flight
Single radio link carries everything	A link loss is a control loss. It has already caused one in-flight failsafe	The USB adapter improved the weak direction. A second independent link removes the single point of failure
Weak, shared-style passwords	Anyone on the aircraft's Wi-Fi who guesses one gets control	Per-aircraft random tokens, rotated before public demonstrations
Fleet not uniform (different JetPack versions)	Behaviour differs between aircraft and tooling silently fails on one of them	Reflash Omega when an x86 Ubuntu machine is available
Knowledge concentrated in one person	The project was already inherited once with no handover	The handoff document, this document, and tests that encode the safety-critical behaviour

Risk	Exposure	What reduces it
Machines running older code than the repository	A fix that exists is not the same as a fix that is deployed	Check the running commit and service directory on each machine before flying or diagnosing

18. Quick reference

Where things live

Path	Machine	Contents
/etc/drone/drone.env	Each aircraft	That aircraft's password and settings. Not in git
/etc/drone/ground.env	Ground	Ground station settings. Not in git
/etc/drone/fleet.json	Ground	The fleet list with health-check tokens. Not in git
~/Documents/ASQI-DRONE	Ground, Omega	The code
ground/gui/dist	Ground	The built web interface, which is what is actually served

Commands used most

Command	What it does
systemctl status ground-bridge ground-detector	Ground station services
systemctl status drone-airborne drone-camera	Aircraft services
journalctl -u drone-airborne -n 200	Recent aircraft log, including every pre-arm message
tools/benchmode.sh on off status	Relax or restore the indoor arming settings, with read-back
tools/params.py get <NAME>	Read a flight controller setting from the vehicle
tools/linkcheck.sh	Live link quality, for the range test

ONE CLOSING NOTE, CARRIED OVER FROM THE HANDOFF DOCUMENT

This document describes the system as it was understood on the date on the cover. It describes the repository and the people's intentions; the machines run whatever they last received. Several times in this project a confident explanation turned out to be wrong, and each time the correction came from reading the vehicle or the logs rather than from re-reading the documentation. Treat everything here as a starting point for that reading, including the settings tables.